

Wedding Moment Nepal

Next.js 16 Storefront — Hardening Review

Code-level audit + deployment hardening plan

8 CRITICAL

VERDICT

Do not deploy to production until all 8 Critical findings are closed and re-verified.

1. Executive Summary

The Wedding Moment Nepal storefront is a custom Next.js 16 application that handles customer personally identifiable information (PII) — names, email addresses, phone numbers, wedding dates — and processes payment initiation through the eSewa and Khalti Nepali payment gateways. A static code review of the entire `src/` tree, the Prisma schema, the deployment scripts, and the Caddy reverse proxy configuration has identified **50 security findings**: 8 Critical, 9 High, 14 Medium, and 19 Low / Informational. Several of these issues are individually critical; together they constitute a systemic security failure that would block any responsible production deployment.

The most urgent findings are: (1) the production Caddyfile contains an unauthenticated reverse-proxy vulnerability that allows any external attacker to probe arbitrary localhost ports on the server (C-8); (2) the `GET /api/bookings` endpoint has no authentication and returns the 50 most recent customer bookings — including PII — to any anonymous caller (C-1); (3) the SQLite database and the `.env` file are both committed to git history with real customer data and configuration (C-5, C-6); (4) the build script copies the development database verbatim into the production tarball (C-7); and (5) when the eSewa or Khalti secret keys are unset, the application silently enters a demo mode that marks orders as paid without any actual payment occurring (C-3).

Beyond the Critical tier, the application lacks every baseline control expected of a payment-handling web app: no input validation framework (Zod is installed but unused), no rate limiting on any endpoint, no CSRF protection, no Content-Security-Policy or other security headers, no authentication system (the `next-auth` dependency is declared but never wired up), no audit log table, and no payment callback routes — meaning that even after a customer pays, the order row is never updated from initiated to paid, because the callback URLs configured in the eSewa/Khalti initiate requests simply return a 404.

Verdict. Do not deploy this build to a production environment. The recommended path is to complete Tier 1 of the remediation roadmap in this report (approximately 4 hours of work) before any further production activity, then proceed through Tier 2 and Tier 3 in the following two weeks. Section 11 of this report contains a written incident-response plan that should be operational before the site accepts real customer payments.

2. Scope & Methodology

In scope. Every source file under `src/` (API routes, components, library code, configuration), the Prisma schema at `prisma/schema.prisma`, the public assets directory, and all root-level configuration files including `next.config.ts`, `Caddyfile`, `package.json`, `.env`, and the build/start scripts under `.zscripts/`. Dependencies declared in `package.json` were checked against current advisory databases for known vulnerabilities.

Out of scope. The runtime container, the network layer beyond the Caddyfile, the CI/CD pipeline (none exists at present), and the external eSewa and Khalti servers themselves. The reviewer did not perform dynamic penetration testing, fuzzing, or runtime exploit verification — all findings are derived from static analysis of the code as committed at `7c7ffec` on the

project's main branch.

Methodology. Each file was read in full. API routes were assessed against the OWASP API Security Top 10 (2023). Payment flows were compared against the official eSewa v2 and Khalti v2 integration specifications. Configuration files were checked against the Next.js and Caddy hardening guides. Findings were graded on a five-level severity scale (Critical / High / Medium / Low / Informational) based on exploitability, blast radius, and the sensitivity of the data at risk. Every finding references its exact file path and line number to enable rapid remediation.

Stack translation note. The user's original request asked for a Shopify/WooCommerce security review. The actual application is neither — it is a custom Next.js storefront. The 10 original checklist items have been translated to fit the real stack, and Appendix A provides the inverse mapping (each Next.js finding → what it would correspond to on Shopify/WooCommerce) in case the studio ever migrates platforms.

3. Stack Overview

The audited application is built on Next.js 16.1.1 with the App Router, using Turbopack as the bundler and `output: "standalone"` for production builds. The database is SQLite, accessed through Prisma ORM 6.11.1, with the database file persisted at `db/custom.db` inside the project root. Payment integration uses the eSewa v2 HMAC-SHA256 signed flow and the Khalti v2 server-to-server ePayment initiation API. The production runtime is Bun (not Node.js), fronted by a Caddy 2 reverse proxy listening on port 81. The frontend uses Tailwind CSS 4 with the shadcn/ui component library, next-themes for dark mode, and Google Fonts (Montserrat body, Playfair Display headings).

The deployment model is unusual: the build script (`.zscripts/build.sh`) produces a tarball that contains the standalone Next.js server, a copy of the development SQLite database, the Caddyfile, and a start script. This tarball is uploaded to the hosting environment and started with `caddy run` as PID 1, which in turn spawns `bun server.js` as a child process. There is no Dockerfile, no infrastructure-as-code, no CI/CD pipeline, and no automated backup mechanism — the SQLite file lives in the container filesystem and is lost if the container is recreated.

4. Original 10-Point Checklist — Translated to Actual Stack

The user's original request contained 10 checklist items framed for Shopify/WooCommerce. The table below maps each item to its equivalent concern in the actual Next.js stack and gives the current status. Detailed findings for each failed item appear in Sections 5–8.

#	Original Checklist Item	Next.js Stack Equivalent	Status
1	HTTPS / SSL enforced site-wide	Caddy TLS termination + HSTS header in next.config.ts	FAIL — no HSTS, no security headers at all (H-4)

#	Original Checklist Item	Next.js Stack Equivalent	Status
2	Audit installed plugins/apps	npm dependencies + custom API routes	FAIL — unused deps, no rate limit, no validation (H-1, H-2, L-6/7/8)
3	Two-factor auth on admin accounts	No User model exists; next-auth declared but unused	FAIL — no auth system whatsoever (C-1, M-7)
4	Daily off-site backups + restore test	SQLite file + cron + S3 sync	FAIL — no backups configured anywhere (Deployment section)
5	Review user roles / remove admin access	Prisma User model + role middleware	FAIL — no users, no roles, no admin panel (M-7)
6	Rate-limiting / brute-force protection	Middleware or Upstash Ratelimit on routes	FAIL — zero rate limiting; no middleware.ts (H-2)
7	Scan for malware / injected code	Repo scan of src/ + public/	PARTIAL — no malware found, but Caddyfile SSRF (C-8) is worse
8	PCI-compliant payment provider	eSewa + Khalti are PCI-compliant gateways	AT RISK — gateways OK, but no callback verification (C-2, C-3, C-4)
9	Activity logging for admin actions	AuditLog Prisma model + middleware	FAIL — no audit log; Prisma logs PII to stdout (H-8, M-7)
10	Written incident-response plan	This report, Section 11	PROVIDED — see Section 11 of this report

Translation principle. Three of the original items (HTTPS, plugin audit, brute-force protection) translate almost directly: HTTPS becomes Caddy TLS plus HSTS in Next.js config; the plugin audit becomes an npm dependency review; brute-force protection becomes rate-limiting middleware. Three items (2FA, user roles, activity logging) cannot be addressed at all in the current build because the application has no concept of a user account — there is no User model, no login form, no session cookie. These items require adding NextAuth plus a Prisma User and AuditLog model before they can even be evaluated. The remaining items (backups, malware scan, PCI compliance, IR plan) are operational rather than code-level and are addressed in Sections 11 and 12.

5. Critical Findings (8)

Critical findings represent issues that allow direct compromise of customer data, direct financial loss, or remote code execution. Each must be closed before any production deployment. Estimated remediation effort for all 8 Critical findings: approximately 6–10 hours of developer time.

CRITICAL

C-1. Unauthenticated GET /api/bookings leaks customer PII

src/app/api/bookings/route.ts:43-57

What's wrong

The GET handler returns the 50 most recent bookings to any caller with no authentication, no API key check, no origin check, and no auth header verification. A grep across the entire `src/` tree confirms zero matches for `getSession`, `signIn`, `next-auth`, or `requireAuth` — authentication has never been wired up anywhere in the application.

Why it matters

Any anonymous internet user can run `curl https://<prod-domain>/api/bookings` and immediately receive a JSON array containing real customer names, email addresses, phone numbers, requested service types, wedding dates, and free-text messages. This is a direct violation of the Nepal Privacy Act 2075 and would constitute a notifiable data breach under GDPR if any European data subjects have submitted bookings. The same exposure applies to any future GET endpoint added to `/api/contact` or `/api/payment`.

Suggested fix

Remove the GET handler, or require an authenticated admin session before returning any data. The `next-auth` dependency is already declared in `package.json` — it just needs to be configured (see M-7 for the User and AuditLog schema additions).

Suggested fix (code)

```
// Option A: delete the GET handler entirely if not needed.
// Option B: gate behind NextAuth admin session.
import { getSession } from 'next-auth';
import { authOptions } from '@lib/auth';

export async function GET(req: NextRequest) {
  const session = await getSession(authOptions);
  if (!session || session.user.role !== 'ADMIN') {
    return NextResponse.json({}, { status: 401 });
  }
  const bookings = await db.booking.findMany({
    orderBy: { createdAt: 'desc' },
    take: 50,
  });
  return NextResponse.json({ success: true, bookings });
}
```

CRITICAL**C-2. Payment
callback routes
do not exist — no
signature
verification,
orders never
marked paid**

src/app/api/payment/esewa/initiate/route.ts:83-84,
src/app/api/payment/khalti/initiate/route.ts:70

What's wrong

Both payment initiation routes configure success/failure callback URLs pointing to /api/payment/esewa/callback and /api/payment/khalti/callback, but no such route files exist. A filesystem search under src/app/api/ returns only bookings, contact, payment/esewa/initiate, and payment/khalti/initiate — no callback directories.

Why it matters

In production, after a customer completes payment, eSewa redirects their browser to the callback URL with a base64-encoded data payload and an HMAC-SHA256 signature. That URL currently returns a 404, so the customer sees a broken page immediately after paying. Worse, the PaymentOrder row in the database is never updated from initiated to paid — the studio has no automated record that payment occurred. Even if a callback route is later added without proper verification, an attacker could forge a callback request and mark any order paid, because the eSewa HMAC signature is currently never checked server-side. Khalti has the same gap: its server-to-server lookup verification step is also missing.

Suggested fix

Implement callback routes for both gateways. For eSewa: verify the HMAC signature using the secret key, then mark the order paid. For Khalti: call the /api/v2/epayment/lookup/ endpoint server-side with the pidx to verify status. Always use crypto.timingSafeEqual for signature comparison to prevent timing attacks.

Suggested fix (code)

```
// src/app/api/payment/esewa/callback/route.ts
import crypto from 'crypto';
import { db } from '@lib/db';

export async function POST(req: NextRequest) {
  const form = await req.formData();
  const data = String(form.get('data') || '');
  const signature = String(form.get('signature') || '');
  const signedFieldNames = String(form.get('signed_field_names') || '');

  // Re-derive the signature from the returned fields in the order eSewa specifies.
  const fields = signedFieldNames.split(',').map(f => `${f}=${form.get(f)}`).join(',');
  const expected = crypto
    .createHmac('sha256', process.env.ESEWA_SECRET_KEY!)
    .update(fields)
    .digest('base64');

  // Constant-time comparison to prevent timing attacks.
  const a = Buffer.from(signature);
  const b = Buffer.from(expected);
  if (a.length !== b.length || !crypto.timingSafeEqual(a, b)) {
    return NextResponse.json({ error: 'Invalid signature' }, { status: 400 });
  }
}
```

```
}

// Decode the data payload, look up the order, mark it paid.
const decoded = JSON.parse(Buffer.from(data, 'base64').toString('utf8'));
await db.paymentOrder.update({
  where: { transactionUuid: decoded.transaction_uuid },
  data: { status: 'paid', rawResponse: data },
});
return NextResponse.redirect(`${process.env.FRONTEND_URL}/payment/success`);
}
```

CRITICAL**C-3. Demo mode
silently marks
orders as paid
without any
payment**

src/app/api/payment/esewa/initiate/route.ts:45-67,
src/app/api/payment/khalti/initiate/route.ts:43-66

What's wrong

When `ESEWA_SECRET_KEY` or `KHALTI_SECRET_KEY` is unset, both routes log a warning and then immediately persist a `PaymentOrder` row with `status: "paid"` and `rawResponse: "demo_mode"`, returning a success response to the client. The current `.env` file contains only `DATABASE_URL` — neither secret is set — so the deployed app is currently in demo mode, meaning every payment is free.

Why it matters

If this build is deployed to production without setting the env vars (and the build script does not enforce their presence), any visitor can pay NPR 1 for a NPR 14,999 wedding package and immediately receive a success confirmation, with the database marking the order paid. The studio would show up to photograph a wedding that was never paid for. Even if the env vars are correctly set in production, the demo branch remains in the code — any future misconfiguration silently disables payment enforcement with no error or alarm.

Suggested fix

Remove the demo branch entirely. If demo mode is needed for local development, gate it behind `NODE_ENV !== 'production'` AND an explicit `PAYMENT_DEMO_MODE=1` flag, and never auto-set status to 'paid' — use status: 'demo' instead. At boot, fail-fast if production env is missing any required secret.

Suggested fix (code)

```
// Remove the demo branch entirely. Fail fast in production.
// src/lib/env.ts (new file)
export function assertProdEnv() {
  if (process.env.NODE_ENV !== 'production') return;
  const required = [
    'DATABASE_URL', 'ESEWA_SECRET_KEY', 'ESEWA_PRODUCT_CODE',
    'KHALTI_SECRET_KEY', 'BACKEND_URL', 'FRONTEND_URL', 'NEXTAUTH_SECRET',
  ];
  for (const k of required) {
    if (!process.env[k]) {
      throw new Error(`Missing required env var: ${k}`);
    }
  }
}
```

```
// Call assertProdEnv() at the top of every payment route handler.
```

CRITICAL**C-4. Server fully trusts client-supplied payment amount**

```
src/app/api/payment/esewa/initiate/route.ts:18-27,  
src/app/api/payment/khalti/initiate/route.ts:16-26
```

What's wrong

Both payment initiation routes read `amount` from the JSON request body and use it directly as the charge amount after only a basic `Number(amount) > 0` check. The server never cross-references the supplied `packageName` against the canonical `SERVICES` list in `src/lib/site.ts` to verify that the amount matches the actual price.

Why it matters

An attacker can submit a request like `{"amount":1, "packageName":"Wedding Photography", "customerName":"x", "customerPhone":"x"}` and pay NPR 1 for a NPR 14,999 wedding package. In production with real eSewa, the customer is charged NPR 1 and the studio is contractually on the hook for a wedding. In demo mode (the current state of the deployed app), the order is marked paid for NPR 1. The same attack applies to Khalti.

Suggested fix

Never trust client-supplied payment amounts. The server should look up the package by a stable key (not by name) and use the canonical price from the server-side `SERVICES` constant. Optionally also enforce idempotency tokens so the amount can't change between quote and charge.

Suggested fix (code)

```
// Look up the package server-side; ignore client-supplied amount.  
import { SERVICES } from '@lib/site';  
  
export async function POST(req: NextRequest) {  
  const { packageKey, customerName, customerPhone } = await req.json();  
  const pkg = SERVICES.find(s => s.key === packageKey);  
  if (!pkg) {  
    return NextResponse.json({ error: 'Unknown package' }, { status: 400 });  
  }  
  const totalAmount = String(pkg.priceFrom); // server-side canonical price  
  // ... proceed with eSewa/Khalti initiation using totalAmount  
}
```

CRITICAL**C-5. SQLite database with customer PII committed to git**

```
db/custom.db (binary file, tracked since initial commit 2a64e5d)
```

What's wrong

The SQLite database file `db/custom.db` is tracked in git. Running `strings db/custom.db | head -80` reveals real-looking PII: names, emails, phone numbers, transaction UUIDs, and payment statuses. The `.gitignore` rule for `*.log` and `.env*` does not retroactively untrack files

that were committed before the rule was added.

Why it matters

Anyone with read access to the repository — future contributors, anyone a remote is pushed to, anyone who purchases the source code from a contractor — has the full customer PII dataset, forever, in git history. Even if the current data is test data, the schema is designed to hold real customer PII, and the workflow commits the DB on every change. If real customer data is ever added and pushed, it cannot be removed without rewriting git history and force-pushing every remote.

Suggested fix

Remove the file from tracking, add an explicit gitignore rule, purge history with git filter-repo or BFG, force-push all remotes, and notify any affected customers. For production, point DATABASE_URL at a path outside the repo (e.g. /var/lib/wedding-moment-nepal/custom.db).

Suggested fix (code)

```
# 1. Stop tracking the file (keeps local copy).
git rm --cached db/custom.db

# 2. Add to .gitignore (explicit, not wildcard).
echo 'db/*.db' >> .gitignore
echo 'db/*.db-journal' >> .gitignore

# 3. Purge ALL historical copies from every commit.
git filter-repo --path db/custom.db --invert-paths
# (or use BFG Repo-Cleaner: java -jar bfg.jar --delete-files custom.db)

# 4. Force-push to overwrite every remote.
git push origin --force --all
git push origin --force --tags

# 5. Rotate any real PII that was committed. Notify affected customers
#   if these are real bookings (required under Nepal Privacy Act 2075).
```

CRITICAL

C-6. .env file committed to git

.env (currently 1 line: DATABASE_URL=...), tracked since initial commit

What's wrong

The .env file is tracked in git. It currently contains only DATABASE_URL, but the .gitignore pattern .env* was added after the file was already committed, so it remains tracked. The pattern of committing .env is already established; the moment anyone adds ESEWA_SECRET_KEY, KHALTI_SECRET_KEY, or NEXTAUTH_SECRET to the file, those secrets ship to the repo.

Why it matters

This is a near-certain future secret leak. The eSewa and Khalti secret keys control real money movement — anyone with the eSewa product code and secret could forge payment confirmations for the studio's account. The NEXTAUTH_SECRET controls session token signing, and its leak would allow an attacker to forge admin sessions. The current state (only DATABASE_URL exposed) is a warning shot, not a stable situation.

Suggested fix

Untrack .env, add explicit gitignore rules, purge history, force-push. Use a secrets manager (Doppler, Vault, AWS SSM) or at minimum an untracked .env.local for development.

Document required env vars in a committed `.env.example` that contains no real values.

Suggested fix (code)

```
# 1. Stop tracking .env.
git rm --cached .env

# 2. Be explicit in .gitignore (don't rely on wildcard).
echo '.env' >> .gitignore
echo '.env.local' >> .gitignore
echo '.env.production' >> .gitignore

# 3. Purge from history.
git filter-repo --path .env --invert-paths
git push origin --force --all

# 4. Create .env.example (committed) with placeholder values:
# DATABASE_URL=file:./db/custom.db
# ESEWA_SECRET_KEY=your_esewa_secret_here
# ESEWA_PRODUCT_CODE=your_merchant_code
# KHALTI_SECRET_KEY=your_khalti_secret_here
# BACKEND_URL=https://weddingmomentnepal.com
# FRONTEND_URL=https://weddingmomentnepal.com
# NEXTAUTH_SECRET=run `openssl rand -base64 32`
# NEXTAUTH_URL=https://weddingmomentnepal.com
```

CRITICAL**C-7. Build script ships the dev database to production**

`.zscripts/build.sh:143-156`

What's wrong

The build script explicitly copies the development SQLite database (the same one committed to git, containing test entries) into the production build tarball, then runs `prisma db push --accept-data-loss` against the copy. The comment in the script reads "将测试环境数据库复制到构建产物中，生产环境直接使用这份数据库" ("Copy the test environment database to the build artifact; production uses this database directly").

Why it matters

Production customers see a database pre-populated with dev test data (Sujan Dai, Jane Smith, Test User, etc.). When production starts writing real bookings, those real bookings layer on top of test data, mixing real and fake PII. Additionally, `db push --accept-data-loss` is destructive — if the schema has changed since the dev DB was created, columns may be dropped silently during build, losing data without warning. The deployed production environment starts in a known-bad state.

Suggested fix

Never copy a dev database into production. The build script should create an empty production DB at build time (`prisma db push` against a fresh file, no `cp`). Better: production should not ship with a DB file at all — `DATABASE_URL` should point at a managed Postgres or a persistent volume mounted at runtime.

Suggested fix (code)

```
# In .zscripts/build.sh – REPLACE the database copy block:

# OLD (delete this):
```

```
# if [ -f './db/custom.db' ]; then
#     mkdir -p "$BUILD_DIR/db"
#     cp -r ./db/. "$BUILD_DIR/db/"
#     DATABASE_URL="file:$BUILD_DIR/db/custom.db" bun run db:push
# fi

# NEW: create an empty production database with the current schema.
mkdir -p "$BUILD_DIR/db"
DATABASE_URL="file:$BUILD_DIR/db/custom.db" bun run db:push
# The production database starts empty. Real data is added by customers.
```

CRITICAL

C-8. Caddyfile has unauthenticated SSRF via XTransformPort query parameter

Caddyfile:2-13

What's wrong

The production Caddyfile contains a route that reads XTransformPort from the query string and reverse-proxies the request to localhost:{query.XTransformPort}. There is no allowlist, no authentication, no validation. Any external request to http://<host>:81/?XTransformPort=<port> causes Caddy to open a connection to that localhost port and pipe the response back.

Why it matters

This is a textbook server-side request forgery (SSRF) and proxy bypass. An attacker can probe every port on localhost (1 through 65535) to map internal services; reach internal-only admin panels, metrics endpoints (Prometheus on :9090), Docker socket proxies, Redis (:6379), Postgres (:5432), cloud metadata services; and bounce requests through Caddy to disguise their origin. The Caddyfile is committed to git and loaded by .zscripts/start.sh:145 as the production proxy. This vulnerability is remotely exploitable by anyone who can reach the server.

Suggested fix

Remove the @transform_port_query block entirely. If port transformation is genuinely needed (it appears to be a sandbox debugging aid), bind to 127.0.0.1 only and require an HMAC-signed token. At minimum, restrict to a known allowlist of ports.

Suggested fix (code)

```
# Caddyfile – DELETE this entire block:
@transform_port_query {
    query XTransformPort=*
}
handle @transform_port_query {
    reverse_proxy localhost:{query.XTransformPort} {
        header_up Host {host}
    }
}

# If port transformation is genuinely needed (it appears to be a sandbox
# debugging aid), bind to 127.0.0.1 only and require an HMAC-signed token:
# @allowed_port {
```

```
#      query XTransformPort=3000
#      expression {http.request.header.X-Debug-Token} eq "{env.DEBUG_TOKEN}"
# }
```

6. High Findings (9)

High findings represent issues that materially weaken the security posture but are not individually exploitable to the point of immediate compromise. They should be closed within the first week of remediation work.

HIGH

H-1. No input validation framework — Zod installed but unused

All four API routes (bookings/route.ts:9-16, contact/route.ts:9-16, esewa/initiate/route.ts:18-25, khalti/initiate/route.ts:16-23)

What's wrong

The package.json declares zod@^4.0.2, but no route imports it. Validation is ad-hoc: `if (!name || !email || !phone || !service || !date)`, followed by `String(name)` coercion. No email format check, no phone normalization, no length cap, no service enum validation, no date parsing.

Why it matters

A booking with email: "not-an-email", phone: "<script>alert(1)</script>", service: "anything-not-in-our-list", date: "1900-13-99", or message: <10 MB string> is accepted and persisted. `String(name)` on an object produces "[object Object]". A 10 MB message will be written to SQLite, allowing trivial database bloat DoS. If any future admin UI renders the message field without escaping, it becomes a stored XSS.

Suggested fix (code)

```
import { z } from 'zod';

const BookingSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email().max(255),
  phone: z.string().regex(/^\+?\d\s-]{6,20}$/),
  service: z.enum(['portrait', 'wedding', 'commercial', 'event']),
  date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/)
    .refine(s => new Date(s) > new Date(), 'Date must be in the future'),
  message: z.string().max(5000).optional(),
});

export async function POST(req: NextRequest) {
  const parsed = BookingSchema.safeParse(await req.json());
  if (!parsed.success) {
    return NextResponse.json({ errors: parsed.error.flatten(), status: 422 });
  }
  const booking = await db.booking.create({ data: parsed.data });
  // ...
}
```

HIGH

H-2. No rate limiting anywhere

Confirmed by `grep` — zero matches for `'rate.?limit|RateLimit|throttle'` in `src/`. No `middleware.ts` exists.

What's wrong

Every API route accepts unlimited requests from any origin. No in-memory token bucket, no Redis-backed limiter, no per-IP throttling. There is no `middleware.ts` file at all — the project has never had any request-level middleware.

Why it matters

`/api/bookings` and `/api/contact` create a DB row on every POST. An attacker can fill the SQLite file to disk in seconds — each row is small but the message field is uncapped (see H-1), so 1 MB per row $\times$ 1000 requests per second = approximately 1 GB per minute. The payment initiation routes create `PaymentOrder` rows on every POST; an attacker can flood the orders table with millions of rows, breaking any future reconciliation. In production mode, each initiate also calls out to the upstream gateway, so an attacker can exhaust the Khalti API rate limit and get the studio's merchant account throttled.

Suggested fix

Install `@upstash/ratelimit` + `@upstash/redis` (or use Vercel KV). Wrap each route with a 5-req-per-minute-per-IP limit. For form spam, also add Cloudflare Turnstile on the booking and contact forms.

Suggested fix (code)

```
// Install: bun add @upstash/ratelimit @upstash/redis
import { Ratelimit } from '@upstash/ratelimit';
import { Redis } from '@upstash/redis';

const limiter = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(5, '1 m'), // 5 req/min per IP
});

export async function POST(req: NextRequest) {
  const ip = req.headers.get('x-forwarded-for')?.split(',')[0] ?? 'anon';
  const { success } = await limiter.limit(ip);
  if (!success) {
    return NextResponse.json({ error: 'Too many requests' }, { status: 429 });
  }
  // ... proceed with handler
}
```

HIGH

H-3. No CSRF protection on POST routes

All POST handlers in `bookings/route.ts:6`, `contact/route.ts:6`, `esewa/initiate/route.ts:16`, `khalti/initiate/route.ts:16`

What's wrong

These are raw `route.ts` handlers (not Next.js Server Actions, which have built-in CSRF). They accept `Content-Type: application/json` from any origin. No Origin/Referer check, no CSRF token.

Why it matters

Today, since there is no auth (see C-1), the worst an attacker can do is flood the DB (covered by H-2). But the moment admin auth is added, any logged-in admin visiting an attacker-controlled page can be CSRF-tricked into creating bookings, deleting data, or initiating refunds. The pattern is set up wrong from day one.

Suggested fix

Either (a) convert handlers to Next.js Server Actions (which auto-emit CSRF tokens), or (b) add an Origin check plus a double-submit cookie or JWT-bound CSRF token for auth'd routes.

Suggested fix (code)

```
// Add Origin check at the top of every POST handler.
const ALLOWED_ORIGINS = new Set([
  'https://weddingmomentnepal.com',
  'http://localhost:3000',
]);

export async function POST(req: NextRequest) {
  const origin = req.headers.get('origin');
  if (!origin || !ALLOWED_ORIGINS.has(origin)) {
    return NextResponse.json({ error: 'Forbidden origin' }, { status: 403 });
  }
  // ... proceed with handler
}
```

HIGH**H-4. No security headers (CSP, HSTS, X-Frame-Options, etc.)**

next.config.ts:1-12 (the entire file – no headers() function)

What's wrong

The Next.js config sets only `output: "standalone"`, `typescript.ignoreBuildErrors: true`, and `reactStrictMode: false`. No security headers are configured anywhere – not in `next.config.ts`, not in `middleware.ts` (which doesn't exist), not in the Caddyfile.

Why it matters

Without CSP, any future stored XSS (e.g., booking message rendered without escaping) executes unrestricted and can steal data or make authenticated requests. Without `X-Frame-Options: DENY`, the site can be clickjacked – an attacker overlays the booking form on top of a fake "Win a free wedding!" button. Without HSTS, a man-in-the-middle on HTTP can downgrade a customer mid-session. Without `X-Content-Type-Options: nosniff`, uploaded files can be MIME-sniffed into executable types.

Suggested fix (code)

```
// next.config.ts – add a headers() function:
const nextConfig = {
  output: 'standalone',
  poweredByHeader: false, // also fixes H-5
  reactStrictMode: true, // also fixes H-6
  typescript: { ignoreBuildErrors: false }, // also fixes H-6
  async headers() {
    return [{
      source: '/(.*)',
      headers: [
        { key: 'Content-Security-Policy',
          value: "default-src 'self'; img-src 'self' data:; " +
            "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; " +
            "font-src 'self' https://fonts.gstatic.com; " +
            "script-src 'self'; frame-ancestors 'none'; " +
            "base-uri 'self'; form-action 'self'; upgrade-insecure-requests" },
        { key: 'Strict-Transport-Security',
          value: 'max-age=63072000; includeSubDomains; preload' },
      ]
    }];
  }
};
```

```
{ key: 'X-Frame-Options', value: 'DENY' },
{ key: 'X-Content-Type-Options', value: 'nosniff' },
{ key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
{ key: 'Permissions-Policy', value: 'geolocation=(), microphone=(), camera=()' },
],
}];
},
};
```

HIGH

H-5. X-Powered-By header not disabled — framework version leaked

`next.config.ts:1-12`

What's wrong

Next.js by default sends X-Powered-By: Next.js on every response. Bun may add its own header. The config does not set `poweredByHeader: false`.

Why it matters

Allows attackers to fingerprint the stack instantly and target known Next.js CVEs (the 14.x and 15.x lines have had several SSRF and cache-poisoning advisories). Combined with the committed package.json, attackers can confirm the exact patch level and look up matching exploits.

Suggested fix

Add `poweredByHeader: false` to `nextConfig` in `next.config.ts` (shown in the H-4 fix). Also strip the header at Caddy as a defense-in-depth measure.

HIGH

H-6. typescript.ignoreBuildErrors: true and reactStrictMode: false

`next.config.ts:6-9`

What's wrong

Type errors don't block production builds — a route could ship with a typo'd field name (`booking.emial` instead of `booking.email`) and silently write `undefined` to the database. Strict mode is off, disabling React's double-effect safety check that catches impure renders leaking data between concurrent users (e.g., a stale closure leaking one user's session into another's render).

Why it matters

Type errors in production code can cause silent data corruption — the most insidious bug class because nothing crashes, but the wrong data is persisted. React strict mode catches impure effects that leak data between users; without it, a coding mistake could expose one customer's booking data to another customer.

Suggested fix

Set `ignoreBuildErrors: false` and `reactStrictMode: true` in `next.config.ts` (shown in the H-4 fix). Fix any resulting type errors before deploying — they are real bugs hiding behind the silence flag.

HIGH

H-7. Khalti route leaks upstream error response to client

`src/app/api/payment/khalti/initiate/route.ts:97-102`

What's wrong

When Khalti's initiate API returns a non-OK response, the route returns `{ success: false, message: 'Khalti initiation failed', details: data }` with status 502, where `data` is the full Khalti API error response.

Why it matters

Khalti's error responses may include merchant ID, internal request IDs, validation details, or partial credential echoes — useful reconnaissance for an attacker probing the integration. Additionally, returning 502 to the client leaves the `PaymentOrder` row in `initiated` status forever, polluting the database with orders that never resolved.

Suggested fix (code)

```
// Log full error server-side; return generic message to client.
if (!resp.ok) {
  console.error('[Khalti] initiate failed:', data);
  // Mark the order as failed so it doesn't linger in 'initiated'.
  await db.paymentOrder.update({
    where: { purchaseOrderId },
    data: { status: 'failed', rawResponse: JSON.stringify(data) },
  });
  return NextResponse.json(
    { success: false, message: 'Payment gateway unavailable' },
    { status: 502 }
  );
}
```

HIGH

H-8. Prisma client logs all queries including parameter values

`src/lib/db.ts:9-11`

What's wrong

`new PrismaClient({ log: ['query'] })` emits every SQL statement with bound parameter values to `stdout`. Every email, phone number, and message string is printed to `stdout` on every request that touches the database.

Why it matters

The `start` script (`package.json:8`) tees server output to `server.log` in the project root, so all PII lands in a file in the working directory. The `dev` script does the same with `dev.log`. Both files contain raw PII. If `NODE_ENV` is unset (misconfigured production), the query logger also runs in production. Log aggregation services (Datadog, Sentry) may ingest and retain this indefinitely.

Suggested fix

Enable query logging only in development. For production, use ['warn', 'error'] only and rely on application-level structured logging when query tracing is needed.

Suggested fix (code)

```
// src/lib/db.ts – only enable query logging in dev.
import { PrismaClient } from '@prisma/client';

const log = process.env.NODE_ENV === 'development'
  ? ['query', 'error', 'warn']
  : ['error', 'warn'];

export const db = new PrismaClient({ log });
```

HIGH**H-9. start script
tees everything
to server.log in
working directory**

package.json:8

What's wrong

"start": "NODE_ENV=production bun .next/standalone/server.js 2>&1 | tee server.log" writes all server output (errors, stack traces, Prisma query logs if H-8 isn't fixed, any console.log from a route that prints an email) to server.log in the project root.

Why it matters

The file persists, is not rotated, is world-readable by default, and on a containerized deploy comes along for the ride in the image layer. server.log is in .gitignore (so it won't be committed), but the runtime exposure remains: anyone with shell access to the container can read it.

Suggested fix

Send logs to stdout only — let the container runtime or systemd journald capture them. Or write to a properly-permissioned /var/log/wedding-moment-nepal/server.log with logrotate. Never write logs inside the app working directory.

7. Medium Findings (14)

Medium findings are real issues that increase risk or maintainability burden but are not immediately exploitable in isolation. They should be addressed within the first month.

MEDIUM

M-1. Predictable eSewa transactionUuid (Math.random, 1000 suffixes per ms)

src/app/api/payment/esewa/initiate/route.ts:28

Use crypto.randomUUID(): const transactionUuid = `order-\${crypto.randomUUID()}`;

MEDIUM

M-2. Khalti purchaseOrderId is even more predictable (no random suffix)

src/app/api/payment/khalti/initiate/route.ts:27

Same fix as M-1: use crypto.randomUUID().

MEDIUM

M-3. No honeypot / captcha on any form

src/components/site/{booking,contact,payment}.tsx

Add Cloudflare Turnstile (free, privacy-respecting) to each form. Verify server-side before persisting. Also add a honeypot field (hidden input named 'website') and reject submissions where it's filled.

MEDIUM

M-4. Booking date field is a raw string with no server-side validation

src/app/api/bookings/route.ts:11, :24

Parse with z.string().date() (ISO 8601) and .refine(s => new Date(s) > new Date(), 'Date must be in the future').

MEDIUM

M-5. Khalti customer_info.email hardcoded to customer@example.com

src/app/api/payment/khalti/initiate/route.ts:77

Collect email in `payment.tsx` and pass it through. Validate with Zod. Khalti sends receipts to this address — currently the studio gets nothing and the customer gets nothing.

MEDIUM**M-6. amount
stored as Float in
Prisma schema**`prisma/schema.prisma:30`

Change to `amount Decimal @db.Decimal(10, 2)`. Floats cannot represent currency exactly; round-off bugs accumulate over time and break reconciliation.

MEDIUM**M-7. No AuditLog
model — no trail
of admin actions
or payment
transitions**`prisma/schema.prisma:1-47`

Add `User`, `AuditLog`, and `WebhookEvent` models (see code block below). Required for PCI-DSS audit trail and Nepal Privacy Act 2075 data subject access requests.

MEDIUM**M-8. No
soft-delete or
data retention
policy**`prisma/schema.prisma:11-46 (no deletedAt columns)`

Add `deletedAt DateTime?` to each model. Use Prisma soft-delete middleware or explicit `where: { deletedAt: null }` filters. Add a cron job to hard-delete rows older than N years.

MEDIUM**M-9.
BACKEND_URL
and
FRONTEND_URL
default to empty
string — silent
breakage**`src/app/api/payment/esewa/initiate/route.ts:30, khalti/initiate/route.ts:44-45`

Fail-fast at boot if these are unset in production. See the `assertProdEnv()` helper in the C-3 fix.

MEDIUM**M-10. ESEWA_PR
ODUCT_CODE
defaults to
EPAYTEST
(eSewa sandbox)**`src/app/api/payment/esewa/initiate/route.ts:29`

Require the env var in production. Never default to a sandbox code. Same fail-fast pattern as M-9.

MEDIUM

**M-11.
String(name)
accepts objects
and arrays — no
schema coercion**

`src/app/api/bookings/route.ts:20-25, contact/route.ts:20-23`

Use Zod (H-1) — it rejects non-string inputs before they reach `String()` coercion. `String({foo: 'bar'})` produces `"[object Object]"`.

MEDIUM

**M-12. No request
body size limit on
POST routes**

All POST handlers (`await req.json()` with no size check)

Read the body with a size cap: `const text = await req.text(); if (text.length > 50_000) return NextResponse.json({error: 'Body too large'}, {status: 413}); const body = JSON.parse(text);`

MEDIUM

**M-13.
rawResponse:
'demo_mode' is a
confusing
sentinel**

`src/app/api/payment/esewa/initiate/route.ts:55, khalti/initiate/route.ts:54`

Use a structured value: `rawResponse: JSON.stringify({ demo: true, ts: Date.now() })`. Or add a `demo Boolean @default(false)` column.

MEDIUM

**M-14. Production
runs on Bun
runtime**

`package.json:8, .zscripts/start.sh:93`

Switch to `node .next/standalone/server.js`. Pin Node 20 LTS or newer. Bun is less battle-tested for production HTTP serving and has had HTTP smuggling advisories in early versions.

M-7 detail: Add these models to `prisma/schema.prisma`

```
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  passwordHash String
  role        String    @default('admin') // admin | staff
  twoFactorEnabled Boolean @default(false)
  twoFactorSecret String?
  createdAt   DateTime  @default(now())
}

model AuditLog {
  id          String    @id @default(cuid())
  actorId     String?
  action      String    // e.g. 'login', 'booking.update', 'payment.refund'
  entityType  String    // 'booking' | 'payment' | 'user' | 'contact'
```

```
    entityId    String?
    metadata    String? // JSON string
    ip          String?
    userAgent   String?
    createdAt   DateTime @default(now())
}

model WebhookEvent {
    id          String    @id @default(cuid())
    gateway     String    // 'esewa' | 'khalti'
    eventType   String
    payload     String    // raw body
    signature   String?
    verified    Boolean   @default(false)
    createdAt   DateTime @default(now())
}
```

8. Low & Informational Findings (19)

Low and Informational findings are cleanup, hygiene, and defense-in-depth items. None is individually exploitable to a serious outcome, but they collectively indicate an immature security posture and should be addressed during normal maintenance.

ID	File	Issue	Fix
L-1	examples/websocket/	Unused websocket server code shipped in repo	Remove or move to separate repo
L-2	tool-results/	HTML dumps of unrelated site committed	git rm --cached; add to .gitignore
L-3	upload/	World-writable (0777) dir with source archives	chmod 755; remove archives; add to .gitignore
L-4	.zscripts/dev.pid	Dev server PID file committed	git rm --cached; add *.pid to .gitignore
L-5	skills/	200+ MB of unrelated ClawHub skills in project root	Move outside project root
L-6	package.json:61	next-auth@4.x unused; carries CVE-2024-4068	Either implement auth (M-7) or remove
L-7	package.json:62	next-intl unused	Remove
L-8	package.json:16-78	Many unused deps: dnd-kit, mdxeditor, recharts, framer-motion, etc.	Run depcheck; remove unused
L-9	src/components/ui/chart.tsx:83	dangerouslySetInnerHTML (unused component, dev-only config)	No action unless wired to user input
L-10	public/robots.txt	Allows all crawlers; combined with C-1, PII could be indexed	Add Disallow: /api/ (also fix C-1)
L-11	eslint.config.mjs:10-44	All safety rules disabled (no-debugger, no-undef, etc.)	Re-enable rules; fix resulting warnings
L-12	.zscripts/start.sh	NEXT_TELEMETRY_DISABLED only set in build, not in start	Export in start.sh too, or set in .env
L-13	prisma/schema.prisma:14-15, :28	Email/phone plaintext and unencrypted at rest	Encrypt PII at app layer with KMS; min 600 perms on DB file
L-14	esewa/initiate/routes.ts:71-73	HMAC key length not validated	Add: if (typeof key !== 'string' key.length < 16) throw
L-15	Contact email (gmail.com)	No SPF/DMARC on contact email; no transport encryption	Use custom domain email with SPF/DMARC
L-16	dev.log, .zscripts/dev.log	Runtime logs contain sandbox internal IP	Ensure excluded from build tarball
L-17	Root directory	No Dockerfile, no CI, no SBOM generation	Add multi-stage Dockerfile + bun audit in CI

ID	File	Issue	Fix
L-18	package.json:10	prisma db push --accept-data-loss in script	Remove --accept-data-loss; switch to prisma migrate
L-19	Caddyfile:1	Caddy listens on :81 (non-standard)	Document topology; ensure TLS at outer proxy

Which Low findings matter most. L-6 (unused next-auth with a known CSRF bypass CVE) and L-13 (PII plaintext at rest) deserve early attention — L-6 because the dependency is in the install graph even though nothing imports it, and L-13 because anyone with filesystem access to the SQLite file reads every customer's email and phone number directly. L-11 (ESLint rules disabled) and L-18 (`--accept-data-loss` in the db push script) are operational landmines that will cause silent data loss sooner or later. The rest are cleanup that can happen during normal maintenance.

9. Prioritized Remediation Roadmap

The 50 findings have been organized into four tiers based on severity and dependency. Each tier should be fully closed and verified before moving to the next.

Tier 1 — Block Production Deployment (Day 0, ~4 hours)

These findings are individually exploitable and have direct customer impact. No production deployment should occur until every Tier 1 item is closed and re-tested. The total estimated effort is approximately 4 hours for an experienced Next.js developer.

Findings in this tier: C-1 (delete unauthenticated GET /api/bookings), C-5 (purge SQLite DB from git history), C-6 (purge .env from git history), C-7 (rewrite build script to not copy dev DB), C-8 (delete Caddy SSRF block).

Verification: (1) `curl https://yoursite.com/api/bookings` returns 404 or 401; (2) `git log --all --full-history -- db/custom.db` returns empty; (3) `git log --all --full-history -- .env` returns empty; (4) build tarball contains an empty `db/custom.db`; (5) `curl 'https://yoursite.com:81/?XTransformPort=22'` returns Caddy's default 404, not an SSH banner.

Tier 2 — Close Critical Payment & Validation Gaps (Week 1, ~2-3 days)

These findings close the remaining Critical-tier payment issues and add the baseline input validation, rate limiting, and security headers that every web app needs. Effort: 2-3 days.

Findings in this tier: C-2 (implement eSewa and Khalti callback routes with signature verification), C-3 (remove demo mode, add fail-fast env validation), C-4 (server-side price lookup), H-1 (add Zod schemas to all routes), H-2 (add Upstash rate limiting), H-3 (add Origin check), H-4 (add security headers).

Verification: (1) end-to-end test payment with eSewa sandbox — order row transitions from initiated to paid only after the customer actually pays; (2) attempting to submit a booking with `email: "not-an-email"` returns 422 with validation errors; (3) sending 10 requests in 1 minute to `/api/bookings` from the same IP returns 429 on the 6th; (4) browser DevTools shows Content-Security-Policy, Strict-Transport-Security, X-Frame-Options headers on every response.

Tier 3 — Add Auth, Audit Log, and Polish (Week 2-3, ~1 week)

These findings add the user authentication system, audit logging, and the remaining High-tier fixes. This tier requires the most new code — adding NextAuth configuration, a User model, an AuditLog model, and wiring up admin routes. Effort: approximately 1 week.

Findings in this tier: M-7 (User, AuditLog, WebhookEvent Prisma models + NextAuth configuration), H-5 (disable X-Powered-By), H-6 (re-enable TypeScript checks and React strict mode), H-7 (sanitize Khalti error responses), H-8 (restrict Prisma query logging to dev), H-9 (remove server.log tee), M-1, M-2 (use `crypto.randomUUID` for order IDs), M-5 (collect customer email in payment flow), M-6 (Decimal instead of Float for amounts).

Verification: (1) admin login flow works with 2FA enabled; (2) every admin action (login, booking update, payment refund) creates an AuditLog row; (3) X-Powered-By header absent from all responses; (4) running `bun run build` with a deliberate type error fails the build; (5) `server.log` is not created in the working directory.

Tier 4 — Hygiene and Cleanup (Month 2, ongoing)

All remaining Medium and Low findings. These are quality-of-life improvements that harden the codebase but don't block any specific feature. Schedule them as a dedicated cleanup sprint or address opportunistically during related work.

Findings in this tier: all remaining M-3, M-4, M-8 through M-14, and all L-1 through L-19. Priority within this tier: L-6 (remove unused next-auth or implement it), L-13 (encrypt PII at rest), L-11 (re-enable ESLint rules), L-18 (switch from db push to migrate).

10. Incident Response Plan

This section addresses the user's checklist item #10: a written incident-response plan with concrete steps to take if a breach or data loss occurs. The plan follows the standard SANS/NIST six-phase lifecycle (Preparation → Identification → Containment → Eradication → Recovery → Lessons Learned) adapted to a small e-commerce storefront operated by a photography studio.

Phase 1 — Preparation (do this now, before any incident)

Designate an Incident Response Lead (default: the studio owner). Compile a contact list with the following entries and store both a digital copy (in a password manager) and a printed copy off-site. Enable Sentry error tracking (free tier covers a small storefront). Configure automated daily off-site backups of `db/custom.db` to an S3-compatible bucket with a 30-day retention. Test a restore once per quarter. Pre-draft the customer notification email template (below) so it can be sent within hours rather than days.

Contact List Template

Role	Organization	Contact	Purpose
Studio Owner / IR Lead	【Owner name】	【Phone】 , 【Email】	Decision authority
Hosting Provider Support	【Provider name】	【Support phone/ticket URL】	Server access, snapshots
eSewa Merchant Support	eSewa	merchantsupport@esewa.com.np, 9801-571111	Pause/inspect payments
Khalti Merchant Support	Khalti	merchantcare@khalti.com, 9801-671111	Pause/inspect payments
Nepal Cyber Bureau	Nepal Police	cyberbureau@nepalpolice.gov.np, 01-4215100	Report cybercrime
Legal Counsel	【Lawyer name】	【Phone】 , 【Email】	Breach notification law
PR / Communications	【Name or agency】	【Phone】 , 【Email】	Customer comms
Insurance Broker	【Broker name】	【Phone】 , 【Email】	Cyber-liability claim

Phase 2 — Identification (the first hour)

Trigger an investigation if any of these occur: Sentry reports a spike in 5xx errors or a specific payment-related error; a customer reports they were charged but saw no confirmation; the studio receives a booking or payment that doesn't match customer communication; an unfamiliar IP appears in access logs hitting `/api/*` routes repeatedly; the database file grows unusually fast (sign of H-2 attack). Begin a written incident log immediately — timestamp every observation, decision, and action. Assign the incident a severity using the matrix below.

Severity Matrix

Severity	Trigger Examples	Response SLA
SEV-1 Critical	Confirmed PII exfiltration OR forged payments OR DB deletion	All-hands response, invoke Phase 3 immediately
SEV-2 High	Suspected breach, gateway abuse, or sustained DDoS	IR Lead convenes within 1 hour
SEV-3 Medium	Single customer impact, isolated bug, no PII exposure	Resolve within 1 business day
SEV-4 Low	Cosmetic issue, near-miss, hardening opportunity	Track and batch-resolve

Phase 3 — Containment (first 60 minutes of a confirmed SEV-1/SEV-2)

Take the site offline immediately by stopping Caddy: `caddy stop` (or `systemctl stop caddy`). Snapshot the current database file before any changes: `cp db/custom.db /tmp/incident-$(date +%s).db`. Preserve all logs: `journalctl -u caddy --since '24 hours ago' > /tmp/incident-caddy.log` and copy `server.log` to a secure location. Rotate all secrets immediately: eSewa secret key, Khalti secret key, NEXTAUTH_SECRET, any DB credentials. Disable the affected payment gateway in the eSewa/Khalti merchant dashboards until eradication is complete. Notify the IR Lead and legal counsel — do NOT notify customers yet (that's Phase 5).

Phase 4 — Eradication (hours to days)

Identify the root cause by correlating logs with the findings in this audit report. Most likely scenarios given the current codebase: (a) the Caddy SSRF (C-8) was used to reach an internal service; (b) the unauthenticated GET `/api/bookings` (C-1) was scraped; (c) a forged payment callback (C-2) marked orders paid without payment; (d) the demo-mode (C-3) was triggered by an env var being unset. Patch the specific vulnerability using the fix code in this report. Re-deploy from a clean build (after Tier 1 of the remediation roadmap is complete). Verify the fix with the test in the relevant finding's 'Suggested fix' section. Do not bring the site back online until the root cause is patched and verified.

Phase 5 — Recovery and Notification

Restore the database from the most recent clean backup if data was modified. Bring the site back online behind a fresh Caddy config and rotated secrets. Monitor Sentry and access logs for 72 hours for re-infection or follow-on attacks. Notify affected customers within 72 hours of confirmation (Nepal Privacy Act 2075 requires notification; GDPR requires 72 hours if any EU data subjects are involved). Use the template below. Notify eSewa and Khalti merchant support if payment data was involved. Notify the Nepal Cyber Bureau if the incident involves active attack rather than misconfiguration. Notify your cyber-liability insurer within the policy's reporting window.

Customer Notification Email Template

Subject: Security Incident Affecting Your Wedding Moment Nepal Account

Dear [Customer Name],

We are writing to inform you of a security incident that may have affected your personal information. On [date], we discovered that [brief description of incident, e.g., 'an unauthorized party accessed our booking database']. Based on our investigation, the following information of yours may have been exposed: [list specific fields – name, email, phone, wedding date, etc.].

We have taken the following steps: [list actions – site taken offline, vulnerability patched, secrets rotated, etc.]. We recommend that you: [list customer actions – change password if you have an account, be alert for phishing, etc.].

We deeply regret this incident and have implemented additional controls to prevent recurrence. If you have questions, please reply to this email or call [studio phone].

Sincerely,
 [Owner name]
 Wedding Moment Nepal

Phase 6 — Post-Mortem (within 7 days of recovery)

Within 7 days of returning to normal operations, complete a written post-mortem covering: (1) timeline of the incident from first detection to recovery; (2) root cause analysis — which finding in this audit report was exploited, and why wasn't it closed; (3) what worked and what didn't in the IR process; (4) specific action items with owners and due dates to prevent recurrence. Share the post-mortem with the IR Lead, legal counsel, and any affected team members. Update this incident-response plan based on lessons learned. File the post-mortem with the incident log for future reference and for any regulatory audit.

11. Deployment Hardening Checklist

Operational hardening that should be in place before the site accepts real customer payments. Each area has a brief explanation and a checklist of concrete items.

Environment Variables

All secrets must be set in the production environment before the application boots. The `assertProdEnv()` helper (see C-3 fix) should fail-fast at startup if any required variable is missing. Never ship secrets in the git repo or in the build tarball.

- ☐ `DATABASE_URL` points to a path outside the repo
- ☐ `ESEWA_SECRET_KEY` set (production value, not `EPAYTEST`)
- ☐ `ESEWA_PRODUCT_CODE` set (production merchant code)
- ☐ `KHALTI_SECRET_KEY` set (production live key)
- ☐ `BACKEND_URL` = `https://weddingmomentnepal.com`
- ☐ `FRONTEND_URL` = `https://weddingmomentnepal.com`
- ☐ `NEXTAUTH_SECRET` = output of `openssl rand -base64 32`
- ☐ `NEXTAUTH_URL` = `https://weddingmomentnepal.com`
- ☐ `UPSTASH_REDIS_REST_URL` and `UPSTASH_REDIS_REST_TOKEN` for rate limiting
- ☐ `SENTRY_DSN` for error tracking
- ☐ `NODE_ENV` = `production`

Backups

Automated daily off-site backups of the SQLite database, with monthly restore tests. Backups must be encrypted at rest in the off-site location and access-controlled. A backup that has never been restored is not a backup — it's a hope.

- ☐ Daily cron job: `sqlite3 db/custom.db '.backup /tmp/backup.db'` then upload to S3-compatible storage
- ☐ Backup retention: 30 days daily, 12 months monthly
- ☐ Backups encrypted with AES-256 (S3 server-side encryption or GPG)
- ☐ Restore test: monthly, restore to a staging DB and run a query
- ☐ Off-site: backups stored in a different region/account than production
- ☐ Access: backup bucket is read-only for the production IAM user
- ☐ Documented: restore procedure is written down and tested by a second person

Monitoring

Three layers of monitoring: errors (Sentry), access logs (Better Stack or Logtail), and transactional email (Postmark or Resend) so the studio knows when bookings/payments occur even if they don't check the admin panel.

- ☐ Sentry: `@sentry/nextjs` installed and configured; DSN in env
- ☐ Sentry alert on any 5xx error in production
- ☐ Access logs: Caddy access log shipped to log aggregator
- ☐ Alert on >100 requests/min from a single IP
- ☐ Alert on any POST to `/api/payment/*/callback` with invalid signature

- [] Transactional email: new booking, new contact, new payment — email to studio
- [] Uptime monitor: external ping every 5 minutes; alert on 3 consecutive failures

Dependency Updates

Next.js has had multiple CVEs in 14.x and 15.x (cache poisoning, SSRF, prototype pollution). Establish a monthly cadence for reviewing and applying updates.

- [] Weekly: `bun audit` in CI; fail build on high or critical
- [] Monthly: review and apply non-security updates
- [] Next.js patch releases: upgrade within 7 days of release
- [] Next.js minor releases: upgrade within 30 days
- [] Subscribe to the Next.js security advisory RSS feed
- [] Subscribe to Prisma security advisories
- [] Quarterly: run `depcheck` and remove unused dependencies

Appendix A — Shopify / WooCommerce Mapping

The user's original request was framed for a Shopify or WooCommerce store. This appendix maps every finding in this report to its equivalent on Shopify and WooCommerce, in case the studio ever migrates platforms. The takeaway: Shopify handles items 1, 4, and 8 essentially for free (TLS, backups, PCI compliance are built-in); WooCommerce handles them with configuration and plugins. The remaining items (2FA, rate limiting, audit logging, incident response) require configuration on both platforms but are easier on Shopify because the admin surface is smaller.

#	Concern	Shopify Equivalent	WooCommerce Equivalent
1	HTTPS / TLS	Automatic on all plans; HSTS managed by Shopify	Requires Let's Encrypt + really-simple-ssl plugin
2	Plugin/app audit	Apps installed via Shopify App Store; review in Admin → Apps	Plugins via WP admin; use Wordfence Scan
3	2FA on admin	Enable in Settings → Users → Two-step authentication	Use Wordfence Login Security or 2FA plugin
4	Daily backups	Automatic; export via Admin → Products → Export	Use UpdraftPlus or Jetpack Backup
5	User roles	Staff accounts with granular permissions; default roles limited	WordPress roles: Subscriber → Super Admin; use Members plugin to customize
6	Rate limiting	Built-in at edge; no configuration needed	Use Wordfence or Cloudflare WAF rule
7	Malware scan	Automatic; Shopify monitors theme files	Use Wordfence Scan or Sucuri
8	PCI compliance	Shopify Payments is PCI-DSS Level 1; no card data on your store	Use Stripe or WooCommerce Payments; never store PANs
9	Activity logging	Limited in core; use Shopify Audit Log (Shopify Plus) or an app	Use WP Activity Log or Wordfence Audit
10	Incident-response plan	Shopify handles infra incidents; you handle store config incidents	Same IR plan as this report; just for WordPress layer

Migration recommendation. If the studio migrates to Shopify, items 1, 4, and 8 are essentially solved by the platform — TLS is automatic, backups are managed, and Shopify Payments is PCI-DSS Level 1 certified. Items 2, 3, and 9 need app installs but are straightforward. Items 5, 6, and 7 still require configuration but are easier because the attack surface is smaller (Shopify admin vs. WordPress + WooCommerce + plugins). Item 10 (incident response) still requires the studio's own plan — this report's Section 11 applies regardless of platform. On WooCommerce, the situation is roughly inverted: the studio gets more control but inherits responsibility for almost every item on the list.

Cost tradeoff. Shopify Basic is approximately USD 39/month plus payment processing fees. WooCommerce is free but requires hosting (USD 10–30/month for a small VPS), an SSL certificate (free with Let's Encrypt), backup plugin (USD 0–15/month), security plugin (USD 0–15/month), and the studio's time to maintain updates. For a small photography studio that does not want to operate infrastructure, Shopify is the lower-risk choice. For a studio that wants full control and has technical capacity, the current Next.js build (after the fixes in this report) is also a reasonable choice — but only after the fixes.

Appendix B — Verification Checklist

A printable checklist the studio owner or a security reviewer can use to confirm each fix is applied. Grouped by finding ID. After all items are checked, complete the Final Go-Live Gate.

Critical Findings

- [] C-1: `curl https://yoursite.com/api/bookings` returns 401 or 404
- [] C-2: End-to-end sandbox payment transitions order from initiated to paid
- [] C-2: Forged callback with wrong signature returns 400
- [] C-3: `grep -r 'demo_mode' src/` returns no matches
- [] C-3: Boot fails fast if `ESEWA_SECRET_KEY` unset in production
- [] C-4: Submitting `{"amount":1, "packageKey":"wedding"}` charges NPR 14999
- [] C-5: `git log --all -- db/custom.db` returns empty
- [] C-6: `git log --all -- .env` returns empty
- [] C-7: Build tarball's `db/custom.db` is empty (0 rows in each table)
- [] C-8: `curl 'https://yoursite.com:81/?XTransformPort=22'` returns 404, not SSH banner

High Findings

- [] H-1: Submitting booking with email: `"not-an-email"` returns 422
- [] H-2: 10 POST requests in 1 minute to `/api/bookings` returns 429 on 6th
- [] H-3: POST from Origin: `https://evil.com` returns 403
- [] H-4: Response headers include CSP, HSTS, X-Frame-Options, X-Content-Type-Options
- [] H-5: `curl -I https://yoursite.com` shows no X-Powered-By
- [] H-6: `bun run build` fails when a type error is introduced
- [] H-7: Khalti error response in browser shows generic message, no `details` field
- [] H-8: Production stdout does not show SQL queries with parameter values
- [] H-9: `server.log` does not exist in working directory after start

Medium Findings (spot-check)

- [] M-1, M-2: `grep 'Math.random' src/app/api/payment/` returns empty
- [] M-5: Payment form collects email; Khalti receipt goes to customer
- [] M-6: `prisma/schema.prisma` uses `Decimal @db.Decimal(10, 2)`
- [] M-7: `prisma/schema.prisma` has User, AuditLog, WebhookEvent models
- [] M-12: POST with 100 KB body returns 413

Final Go-Live Gate

Before accepting real customer payments, the following five items must all be confirmed. If any is unchecked, do not go live.

- ☐ 1. All 8 Critical findings closed and verified per the tests above
- ☐ 2. End-to-end sandbox payment with both eSewa and Khalti succeeds (customer pays, order row transitions to paid, customer sees thank-you page)
- ☐ 3. Daily backup configured and a test restore has been performed successfully
- ☐ 4. Sentry error monitoring is live and receiving events
- ☐ 5. The incident-response plan in Section 11 has been read by the studio owner and the contact list (Section 11, Phase 1) has been filled in